

University of Notre Dame

Department of Computer Science and
Engineering

CSE 60745

Graph Learning: Techn & Apps

Assignment Date:	
Submission Date:	09/14/2026
Student Name:	

Homework 1 — Graph Basics and Properties

Part A: Graph Creation and Modification (30 pts)

Part B: Graph Essentials and Measures (70 pts)

Total: 100 pts

Part A – Graph Creation and Modification (30 pts)

Q1: Graph Creation (15 pts)

The graph shown in the assignment is an undirected graph on the ten vertices $V = \{0, 1, \dots, 9\}$. Reading the edges off the figure gives $|E| = 15$ edges:

$$E = \left\{ \begin{array}{l} (0, 1), \quad (0, 4), \quad (0, 5), \quad (1, 2), \quad (1, 6), \\ (2, 3), \quad (2, 7), \quad (3, 4), \quad (3, 8), \quad (4, 9), \\ (5, 7), \quad (5, 8), \quad (6, 8), \quad (6, 9), \quad (7, 9). \end{array} \right\}$$

Equivalently, as an adjacency list:

Node	Neighbours	Node	Neighbours
0	1, 4, 5	5	0, 7, 8
1	0, 2, 6	6	1, 8, 9
2	1, 3, 7	7	2, 5, 9
3	2, 4, 8	8	3, 5, 6
4	0, 3, 9	9	4, 6, 7

Construction. The graph was built in NetworkX by creating an empty undirected graph, adding the ten nodes $0, \dots, 9$, and then adding the fifteen edges listed above:

```
G = nx.Graph()
G.add_nodes_from(range(10))
G.add_edges_from(E)
```

Checks on the transcription. Every vertex has degree exactly 3, so the graph is 3-regular and the handshake lemma is satisfied: $\sum_v \deg(v) = 30 = 2|E| = 2 \cdot 15$. The graph is connected, has no triangles, and has girth 5. In fact these fifteen edges reproduce the *Petersen graph* (checked with NetworkX's `is_isomorphic` against `petersen_graph()`, which returns `True`), the standard 3-regular, girth-5 graph on ten vertices. Landing on the Petersen graph is a strong confirmation that the edge list was read off the drawing correctly.

One edge deserves comment, because the drawing in the assignment is misleading at that point: vertices 5, 4 and 8 are very nearly collinear, and the single edge $(5, 8)$ is drawn passing straight across the disc of vertex 4. It therefore *looks* like a pair of edges $(5, 4)$ and $(4, 8)$, but it is one edge; if it were two, vertices 5, 4 and 8 would all have degree 4 and the graph would not be 3-regular. In Figure 1 the edge $(5, 8)$ is drawn with a slight bow so that it visibly passes clear of vertex 4.

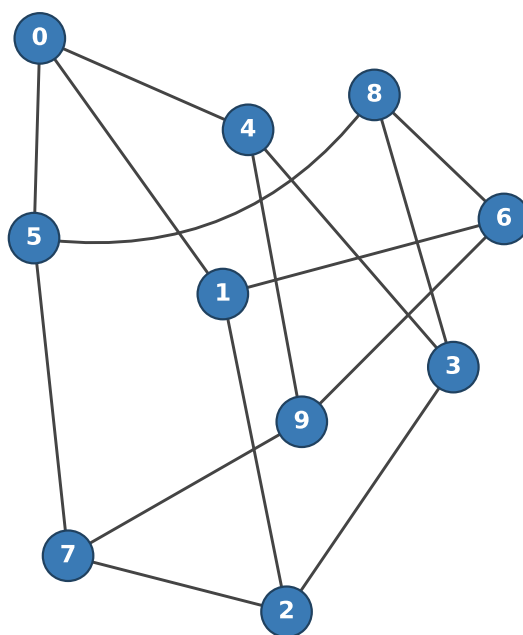


Figure 1: Q1: the undirected graph on 10 vertices and 15 edges, drawn with the same vertex placement as the figure in the assignment. The edge $(5, 8)$ is bowed so that it is visibly distinct from the vertex 4 it passes behind.

The visualization is shown in Figure 1. It uses the same vertex positions as the drawing in the assignment, so the two can be compared directly.

Q2: Graph Modification (15 pts)

Starting from the graph G of Q1 we remove, in order,

1. **node** 1, which also deletes its three incident edges $(0, 1)$, $(1, 2)$ and $(1, 6)$;
2. **edge** $(0, 4)$;
3. **edge** $(0, 5)$.

Five edges are removed in total, leaving the graph H on nine vertices with ten edges:

$$E(H) = \{(2, 3), (2, 7), (3, 4), (3, 8), (4, 9), (5, 7), (5, 8), (6, 8), (6, 9), (7, 9)\}.$$

	Before (Q1)	After (Q2)
Nodes	10	9
Edges	15	10
Connected components	1	2

Remaining components. The three edges at node 0 were exactly $(0, 1)$, $(0, 4)$ and $(0, 5)$, and all three are removed by the three operations above. Node 0 is therefore left isolated, and the resulting graph has **two** connected components:

Component	Vertices	Size
C_1	$\{0\}$	1
C_2	$\{2, 3, 4, 5, 6, 7, 8, 9\}$	8

C_2 contains all ten remaining edges and is connected; C_1 is the isolated vertex 0. The remaining graph is drawn in Figure 2, using the same vertex positions as Figure 1 so that the effect of the modification is directly visible; the isolated vertex 0 is drawn in orange.

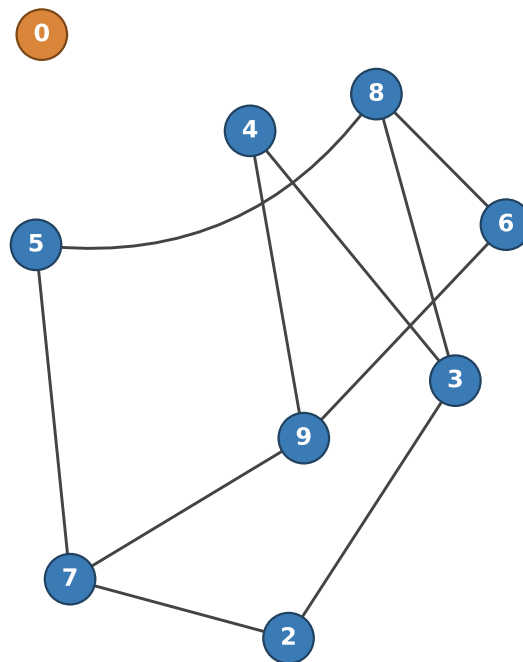


Figure 2: Q2: the graph after removing node 1 and the edges $(0, 4)$ and $(0, 5)$. Vertex positions are unchanged from Figure 1. The two connected components are the isolated vertex 0 (orange) and the remaining eight vertices.

Part B – Graph Essentials and Measures (70 pts): Q3 and Q4

Q3: Graph Creation (10 pts)

The edge list `graph.txt` (attached to this assignment as `graph-1.txt`) contains 26,377 tab-separated lines, each giving one undirected edge. It was loaded into a simple undirected graph (`networkx.Graph`), adding one node per distinct identifier appearing in the file and one edge per line; no isolated nodes were added and the file was not symmetrised a second time. The file contains no self-loops and no duplicate edge in either orientation, so the 26,377 lines yield exactly 26,377 distinct edges.

Quantity	Value
Number of nodes	26,728
Number of edges	26,377
Number of connected components	4,104

Table 1: Q3: basic counts for the graph built from the edge list.

Answer. The graph has **26,728 nodes**, **26,377 edges** and **4,104 connected components**.

The graph is therefore very sparse (mean degree $2m/n \approx 1.97$) and highly fragmented. It is not a forest: a forest on n nodes with c components would have $n - c = 26,728 - 4,104 = 22,624$ edges, whereas the graph has 26,377, so it carries $m - n + c = 3,753$ independent cycles.

Q4(a): Largest connected component (10 pts)

Connected components were extracted with `nx.connected_components` and sorted by size in decreasing order; the largest is the first element. Its edge count is that of the *induced* subgraph on those nodes, not the edge count of the whole graph.

Component	Nodes	Edges
Largest (LCC)	13,903	16,695
Second largest	69	80
Whole graph	26,728	26,377

Table 2: Q4: size of the two largest connected components, against the whole graph.

Answer. The largest connected component has **13,903 nodes** and **16,695 edges**. It is a giant component in the usual sense: it holds 52.0% of all nodes and 63.3% of all edges, while the remaining 4,103 components share the other half of the nodes.

Q4(b): Visualization of the second largest connected component (10 pts)

Which component, and how it was selected. The components were sorted by node count in decreasing order and the element at index 1 was taken, i.e.

```
sorted(nx.connected_components(G), key=len, reverse=True)[1].
```

The ten largest component sizes are

13,903, **69**, 30, 30, 29, 26, 23, 21, 21, 20.

There is no tie for second place: exactly one component has 69 nodes, so the choice is unambiguous and no tie-breaking rule was needed. (The tie that does exist is at *third* place, where two distinct components both have 30 nodes; it does not affect this question.) The selected component has **69 nodes** and **80 edges**.

Figure 3 shows it. The layout is Kamada–Kawai, which is deterministic for a given graph (circular initialisation followed by BFGS energy minimisation), so the drawing is reproducible. Node area encodes degree and node colour repeats it via the colour bar. **Node identifiers are deliberately not drawn:** with 69 five-digit identifiers the labels overlap each other and are occluded by the markers in the dense core of the component, which makes the picture less readable rather than more. The structure is a sparse, tree-like skeleton with a few short cycles — consistent with $80 - 69 + 1 = 12$ independent cycles — with two hubs of degree 7 and 6 near the centre and a long chain of degree-2 nodes reaching to the upper right (the component has diameter 17).

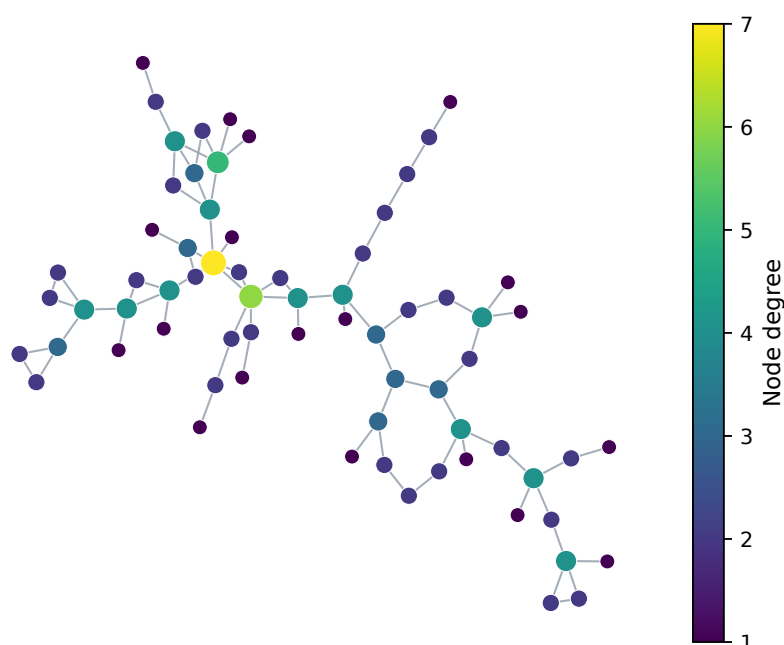


Figure 3: Q4(b): the second largest connected component (69 nodes, 80 edges), drawn with a Kamada–Kawai layout. Node size and colour both encode degree. Node labels are omitted for legibility.

Q4(c): Degree distribution of the whole graph (10 pts)

Figure 4 plots the degree distribution of the *entire* graph (all 26,728 nodes, not just the giant component). The left panel gives the raw node count $N(k)$ on linear axes; the right panel gives the fraction of nodes $P(k) = N(k)/n$ on log–log axes, which is the informative view for a sparse graph with a heavy tail.

Answer. The degree ranges from a **minimum of 1** to a **maximum of 59**, with **mean degree 1.974** ($= 2m/n = 2 \times 26,377/26,728$). No node is isolated, since every node enters the graph through an edge.

The distribution is strongly right-skewed. Degree 1 alone accounts for 13,942 nodes (52.2%) and degree 2 for a further 7,423 (27.8%), so four fifths of the graph has degree at most 2; the median degree is 1. Only 37 distinct degree values occur at all, and the tail is thin and ragged — the

Degree statistic	Value
Minimum degree	1
Maximum degree	59
Mean degree	1.974
Median degree	1

Table 3: Q4(c): degree statistics over all 26,728 nodes.

largest degrees (36, 38, 40, 51, 57, 59) are each realised by a single node, the maximum being node 22777 with degree 59. On log-log axes the distribution falls off approximately linearly over the bulk of its range before scattering in the tail, which is the signature of a heavy-tailed, roughly power-law-like degree distribution rather than a Poisson one. This is also why the linear panel alone is misleading: on linear axes the whole distribution collapses into a single spike at $k \leq 2$ and the tail out to $k = 59$ is invisible.

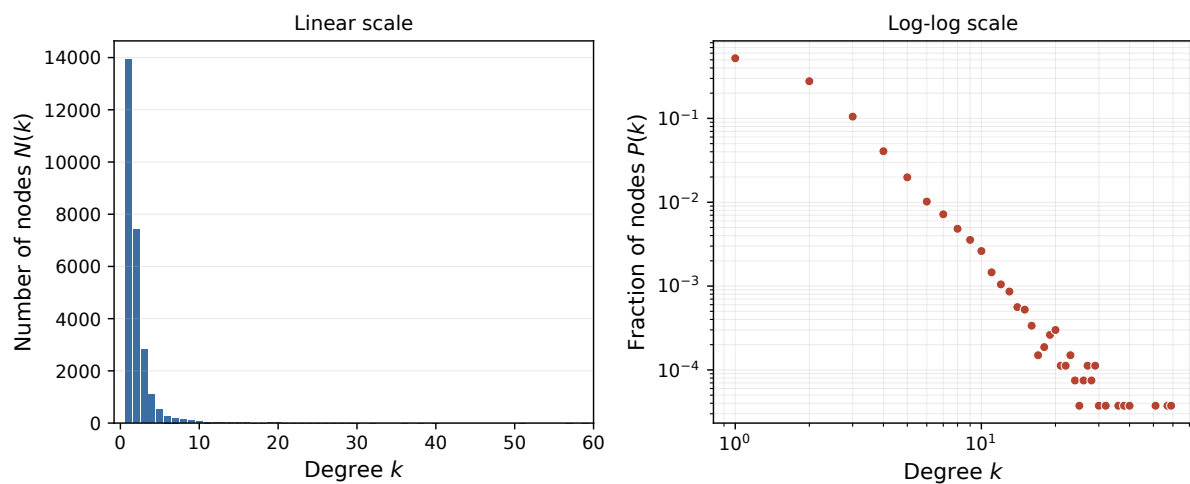


Figure 4: Q4(c): degree distribution of the whole graph. Left: node count $N(k)$ against degree k on linear axes. Right: the same distribution as a fraction of nodes $P(k)$ on log-log axes, which reveals the heavy tail out to $k = 59$.

Part B – Graph Essentials and Measures (70 pts): Q5

All measures below are computed on the graph built in Q3: the simple undirected graph read from `graph.txt` (one `networkx.Graph`, one node per distinct id, one edge per line), which has 26,728 nodes, 26,377 edges and 4104 connected components, the largest of which contains 13,903 nodes and 16,695 edges. All computations use NetworkX with its default parameters; every number was reproduced under NetworkX 3.3 and 3.6.1 and independently re-derived with SciPy (see the remarks).

Q5(a): Eigenvector centrality (10 pts)

`nx.eigenvector_centrality(G)` was called with default parameters (`max_iter= 100`, `tol= 10-6`). **It converged**; no `PowerIterationFailedConvergence` was raised, so the default call is the one reported. Values are the entries of the principal eigenvector of the adjacency matrix, normalised to unit L^2 norm.

Rank	Node	Degree	Centrality (default)	Centrality (exact)
1	16485	57	0.491844	0.492371
2	22777	59	0.419863	0.422581
3	14618	38	0.177656	0.177494
4	21928	17	0.139183	0.139695
5	2884	29	0.133813	0.133861
6	1197	18	0.133186	0.133738
7	9068	51	0.111138	0.102654
8	17902	32	0.109219	0.109103
9	1324	20	0.100115	0.100053
10	25747	19	0.094368	0.094399

Table 4: Q5(a): the ten nodes with the largest eigenvector centrality.

Remark on the default tolerance. The fourth column is the answer with default parameters, as asked. For comparison the fifth column is the exact eigenvector from `eigenvector_centrality_numpy`. The two agree on the *set* of top-10 nodes, but they order ranks 7 and 8 differently: the exact computation puts node 17902 ahead of node 9068. The reason is that NetworkX’s power iteration stops when the total L^1 change falls below

$$n \cdot \text{tol} = 26,728 \times 10^{-6} \approx 2.67 \times 10^{-2},$$

which is a loose *absolute* bound on a vector of unit norm; the returned vector has residual $\|Ax - \lambda x\|_\infty \approx 7.8 \times 10^{-3}$ and Rayleigh quotient 8.76202 against the true $\lambda_{\max} = 8.762217$. Node 9068 is the entry still furthest from its limit, so its rank has not settled after 100 iterations. Running the same power iteration with `max_iter= 105`, `tol= 10-10` reproduces the exact column to 9.3×10^{-7} .

Interpretation. Because the graph is disconnected, the principal eigenvector is supported on the component with the largest spectral radius. Here that is the giant component ($\lambda_{\max} = 8.7622$, versus at most 3.9193 for every other component), so all ten top nodes lie in the giant component and roughly half the nodes (12,908, or 48.3%) receive centrality below 10^{-12} . Note also that eigenvector centrality is not simply degree: only 6 of these 10 nodes are also in the top 10 by degree. Node 21928 ranks 4th with degree just 17, because its neighbours are themselves central, while node 17546 has degree 40 yet does not make the list.

Q5(b): Katz centrality (10 pts)

`nx.katz_centrality(G)` was called with default parameters ($\alpha = 0.1$, $\beta = 1.0$, `max_iter`=1000, `tol`= 10^{-6}), solving $x = \beta \mathbf{1} + \alpha Ax$ and normalising to unit L^2 norm.

Admissibility of the default α . Katz centrality is well defined only for $\alpha < 1/\lambda_{\max}$. For this graph $\lambda_{\max} = 8.762217$, so $1/\lambda_{\max} = 0.114126$. The default $\alpha = 0.1$ satisfies $0.1 < 0.114126$, so it **is** admissible (though not by a large margin). Consistently with this, the default call **converged** and every entry of the unnormalised solution is positive.

Rank	Node	Degree	Katz centrality
1	16485	57	0.211609
2	22777	59	0.175512
3	9068	51	0.109482
4	14618	38	0.099380
5	2884	29	0.074102
6	17902	32	0.070590
7	17546	40	0.068771
8	21928	17	0.066834
9	1197	18	0.061642
10	9740	27	0.060545

Table 5: Q5(b): the ten nodes with the largest Katz centrality.

These values were confirmed against `nx.katz_centrality_numpy(G)` and against a direct sparse solve of $(I - \alpha A)x = \beta \mathbf{1}$; all three give the same ordering, agreeing to 1.3×10^{-5} and 1.9×10^{-16} respectively. Unlike eigenvector centrality, the $\beta \mathbf{1}$ term gives every node a positive baseline, so Katz does not collapse to zero off the giant component, and its ranking tracks degree more closely (node 9068, degree 51, rises from 7th to 3rd).

Q5(c): Average clustering coefficient and average Jaccard similarity (10 pts)

Average clustering coefficient. Using `nx.average_clustering(G)`, i.e. the mean over *all* 26,728 nodes of the local clustering coefficient $c_u = 2T_u/(d_u(d_u - 1))$, where T_u is the number of triangles through u , with the standard convention that $c_u = 0$ for nodes of degree < 2 :

$$\bar{C} = 0.100539.$$

This average is held down by the graph's sparsity: 13,942 of the 26,728 nodes (52.2%) have degree < 2 and so contribute 0 by convention, and the graph contains only 1926 triangles in total. Restricted to the 12,786 nodes of degree ≥ 2 the mean local clustering is 0.210168.

Average Jaccard similarity of all connected node pairs. For every edge $(u, v) \in E$ the Jaccard similarity of the two neighbourhoods is

$$J(u, v) = \frac{|N(u) \cap N(v)|}{|N(u) \cup N(v)|},$$

computed with `nx.jaccard_coefficient(G, G.edges())` and averaged over all 26,377 edges. Here $N(\cdot)$ denotes the *raw* neighbour set, so $v \in N(u)$ and $u \in N(v)$; consequently an isolated

edge (both endpoints of degree 1) has $|N(u) \cap N(v)| = 0$ and scores $J = 0$ rather than being undefined. Under this convention

$$\bar{J} = 0.041304.$$

The value is small because 21,167 of the 26,377 edges (80.2%) join two nodes with no common neighbour and therefore score exactly 0; the largest value attained by any edge is $2/3$. (For contrast, excluding the two endpoints from each other's neighbour sets would give 0.084305; the number reported above is the NetworkX convention stated in the formula.)